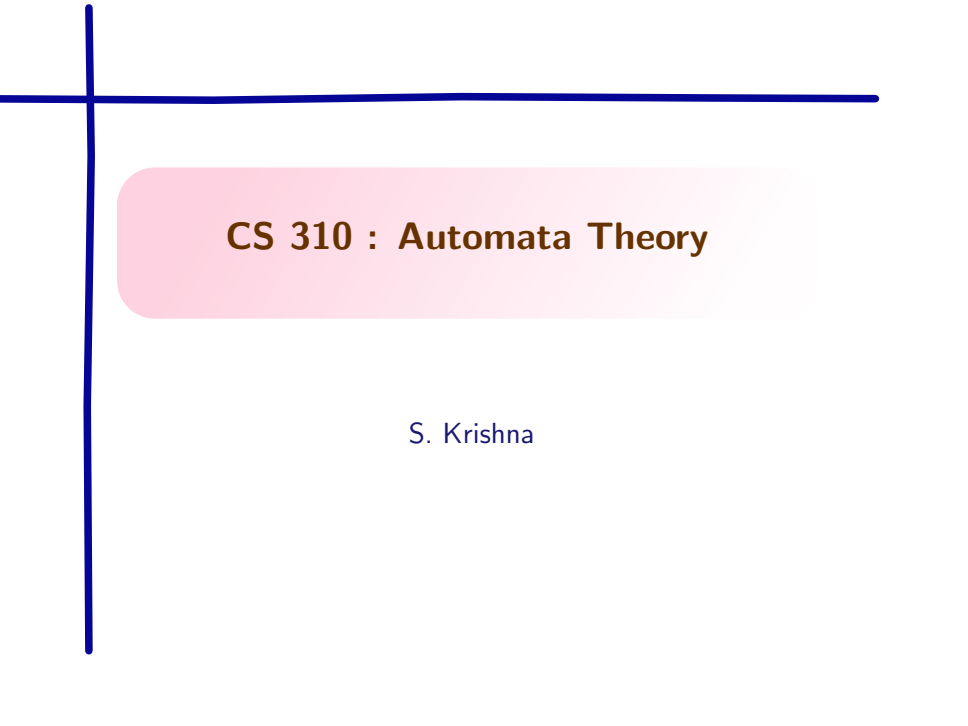


A decorative blue crosshair consisting of a vertical line and a horizontal line intersecting at the top-left corner of the slide.

CS 310 : Automata Theory

S. Krishna

Going Beyond Regularity

Module 2: Finite State Automata with a Stack

Many ways to extend DFA/NFA with additional storage

- ▶ counters
- ▶ registers
- ▶ stacks (LIFO property)
- ▶ queues (FIFO property)

Module 2: Finite State Automata with a Stack

Many ways to extend DFA/NFA with additional storage

- ▶ counters
- ▶ registers
- ▶ stacks (LIFO property)
- ▶ queues (FIFO property)

Anything else?

Module 2: Finite State Automata with a Stack

Many ways to extend DFA/NFA with additional storage

- ▶ counters
- ▶ registers
- ▶ stacks (LIFO property)
- ▶ queues (FIFO property)

Anything else?

We consider one specific case

Pushdown automata: NFA with one (LIFO-)stack

Pushdown automaton (PDA)

Definition

A (non-deterministic) pushdown automaton (PDA) P is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

Pushdown automaton (PDA)

Definition

A (non-deterministic) pushdown automaton (PDA) P is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where

- ▶ Q is a finite set of states,
- ▶ Σ is a finite input alphabet,
- ▶ δ
- ▶ $q_0 \in Q$ is the start/initial state,
- ▶ $F \subseteq Q$ is a set of accepting states.

Pushdown automaton (PDA)

Definition

A (non-deterministic) pushdown automaton (PDA) P is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where

- ▶ Q is a finite set of states,
- ▶ Σ is a finite input alphabet,
- ▶ Γ is a finite set of stack symbols, i.e., stack alphabet
- ▶ δ
- ▶ $q_0 \in Q$ is the start/initial state,
- ▶ $F \subseteq Q$ is a set of accepting states.

Pushdown automaton (PDA)

Definition

A (non-deterministic) pushdown automaton (PDA) P is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where

- ▶ Q is a finite set of states,
- ▶ Σ is a finite input alphabet,
- ▶ Γ is a finite set of stack symbols, i.e., stack alphabet
- ▶ δ
- ▶ $q_0 \in Q$ is the start/initial state,
- ▶ $\perp \in \Gamma$ is the start stack symbol, and
- ▶ $F \subseteq Q$ is a set of accepting states.

Pushdown automaton (PDA)

Definition

A (non-deterministic) pushdown automaton (PDA) P is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where

- ▶ Q is a finite set of states,
- ▶ Σ is a finite input alphabet,
- ▶ Γ is a finite set of stack symbols, i.e., stack alphabet
- ▶ $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ is a transition function,
- ▶ $q_0 \in Q$ is the start/initial state,
- ▶ $\perp \in \Gamma$ is the start stack symbol, and
- ▶ $F \subseteq Q$ is a set of accepting states.

Pushdown automaton (PDA)

Definition

A (non-deterministic) pushdown automaton (PDA) P is a tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where

- ▶ Q is a finite set of states,
- ▶ Σ is a finite input alphabet,
- ▶ Γ is a finite set of stack symbols, i.e., stack alphabet
- ▶ $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ is a transition function,
- ▶ $q_0 \in Q$ is the start/initial state,
- ▶ $\perp \in \Gamma$ is the start stack symbol, and
- ▶ $F \subseteq Q$ is a set of accepting states.

Nondeterministic

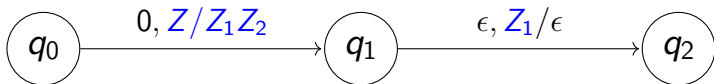
Transitions in pushdown automaton

Let $\Sigma = \{0, 1\}$, $\Gamma = \{\perp, Z, Z_1, Z_2, \dots\}$ and $\delta(q_0, 0, Z) \ni (q_1, Z_1 Z_2)$,
 $\delta(q_1, \epsilon, Z_1) \ni (q_2, \epsilon)$

Transitions in pushdown automaton

Let $\Sigma = \{0, 1\}$, $\Gamma = \{\perp, Z, Z_1, Z_2, \dots\}$ and $\delta(q_0, 0, Z) \ni (q_1, Z_1 Z_2)$,
 $\delta(q_1, \epsilon, Z_1) \ni (q_2, \epsilon)$

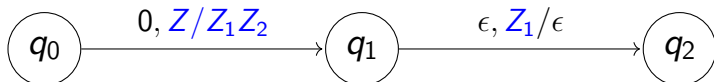
We draw the PDA transitions as follows



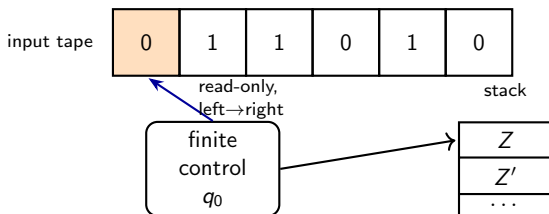
Transitions in pushdown automaton

Let $\Sigma = \{0, 1\}$, $\Gamma = \{\perp, Z, Z_1, Z_2, \dots\}$ and $\delta(q_0, 0, Z) \ni (q_1, Z_1 Z_2)$,
 $\delta(q_1, \epsilon, Z_1) \ni (q_2, \epsilon)$

We draw the PDA transitions as follows



To execute the first transition, top symbol of the stack must be Z ; After transition Z will be popped and word $Z_1 Z_2$ will be pushed

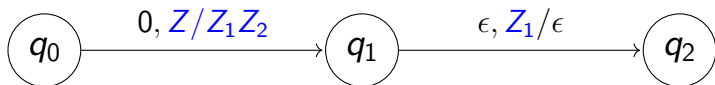


Before: reading 0 in state q_0 , top of stack is Z .

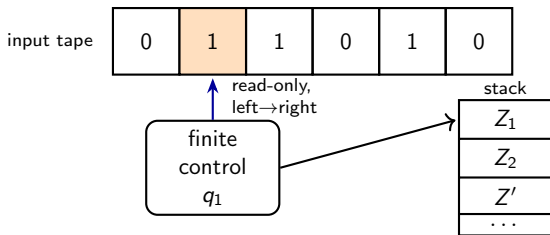
Transitions in pushdown automaton

Let $\Sigma = \{0, 1\}$, $\Gamma = \{\perp, Z, Z_1, Z_2, \dots\}$ and $\delta(q_0, 0, Z) \ni (q_1, Z_1Z_2)$,
 $\delta(q_1, \epsilon, Z_1) \ni (q_2, \epsilon)$

We draw the PDA transitions as follows



To execute the first transition, top symbol of the stack must be Z ; After transition Z will be popped and word Z_1Z_2 will be pushed



After: the 0 is consumed, the pointer moves to 1, Z is popped and Z_1Z_2 is pushed.

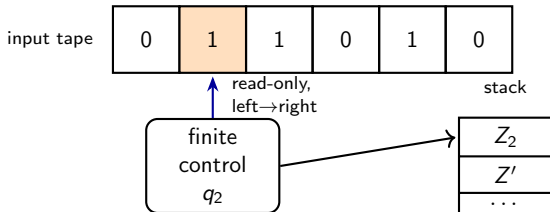
Transitions in pushdown automaton

Let $\Sigma = \{0, 1\}$, $\Gamma = \{\perp, Z, Z_1, Z_2, \dots\}$ and $\delta(q_0, 0, Z) \ni (q_1, Z_1 Z_2)$,
 $\delta(q_1, \epsilon, Z_1) \ni (q_2, \epsilon)$

We draw the PDA transitions as follows



To execute the second transition, no input symbol is read; top symbol of the stack must be Z_1 . After the transition Z_1 will be popped and nothing pushed, so the pointer stays where it is



Understanding stack interaction due to δ

- Popping Z in stack :

$$\delta(q_1, a, Z) \ni (q_2, \epsilon)$$

Understanding stack interaction due to δ

- ▶ Popping Z in stack :

$$\delta(q_1, a, Z) \ni (q_2, \epsilon)$$

- ▶ Pushing X in stack :

$$\delta(q_1, a, Z) \ni (q_2, XZ)$$

Understanding stack interaction due to δ

- Popping Z in stack :

$$\delta(q_1, a, Z) \ni (q_2, \epsilon)$$

- Pushing X in stack :

$$\delta(q_1, a, Z) \ni (q_2, XZ)$$

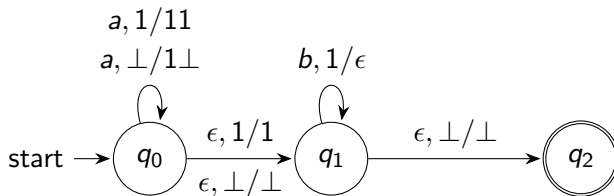
How to model unchanged stack content?

Formal definition of acceptance, later. First some examples.

Example : PDA

PDA recognizing $\{a^n b^n \mid n \geq 0\}$.

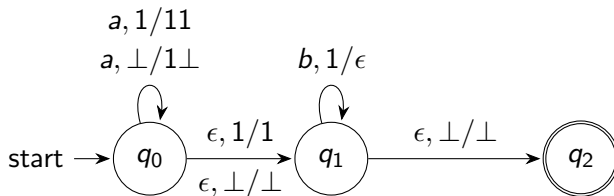
$A = (\{q_0, q_1, q_2\}, \{a, b\}, \{\perp, 1\}, \delta, q_0, \perp, \{q_2\})$, where δ is illustrated as follows



Example : PDA

PDA recognizing $\{a^n b^n \mid n \geq 0\}$.

$A = (\{q_0, q_1, q_2\}, \{a, b\}, \{\perp, 1\}, \delta, q_0, \perp, \{q_2\})$, where δ is illustrated as follows



Exercises

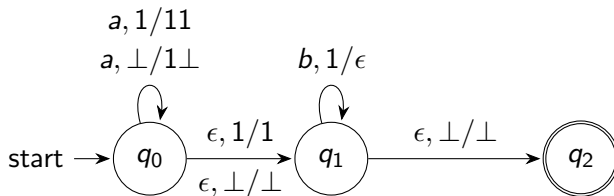
What is the value of the transition function?

► $\delta(q_0, b, 1) =$

Example : PDA

PDA recognizing $\{a^n b^n | n \geq 0\}$.

$A = (\{q_0, q_1, q_2\}, \{a, b\}, \{\perp, 1\}, \delta, q_0, \perp, \{q_2\})$, where δ is illustrated as follows



Exercises

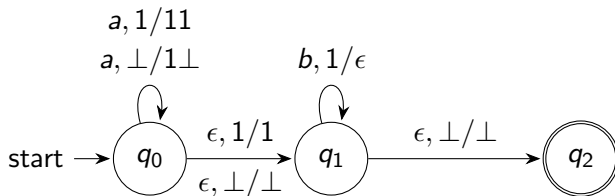
What is the value of the transition function?

- ▶ $\delta(q_0, b, 1) = \emptyset$
- ▶ $\delta(q_1, \epsilon, \perp) =$

Example : PDA

PDA recognizing $\{a^n b^n | n \geq 0\}$.

$A = (\{q_0, q_1, q_2\}, \{a, b\}, \{\perp, 1\}, \delta, q_0, \perp, \{q_2\})$, where δ is illustrated as follows



Exercises

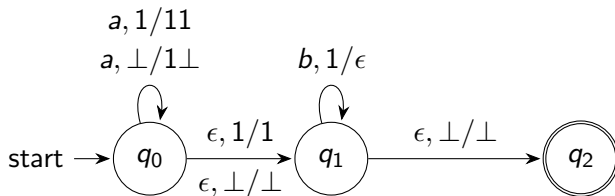
What is the value of the transition function?

- ▶ $\delta(q_0, b, 1) = \emptyset$
- ▶ $\delta(q_0, a, \perp) =$
- ▶ $\delta(q_1, \epsilon, \perp) = \{(q_2, \perp)\}$

Example : PDA

PDA recognizing $\{a^n b^n | n \geq 0\}$.

$A = (\{q_0, q_1, q_2\}, \{a, b\}, \{\perp, 1\}, \delta, q_0, \perp, \{q_2\})$, where δ is illustrated as follows



Exercises

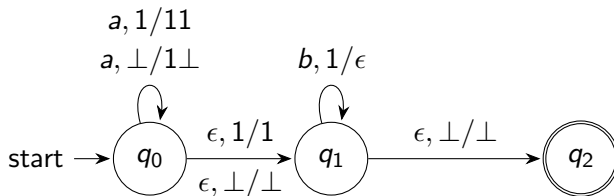
What is the value of the transition function?

- ▶ $\delta(q_0, b, 1) = \emptyset$
- ▶ $\delta(q_0, a, \perp) = \{(q_0, 1\perp)\}$ *push*
- ▶ $\delta(q_1, \epsilon, \perp) = \{(q_2, \perp)\}$
- ▶ $\delta(q_1, b, 1) =$

Example : PDA

PDA recognizing $\{a^n b^n | n \geq 0\}$.

$A = (\{q_0, q_1, q_2\}, \{a, b\}, \{\perp, 1\}, \delta, q_0, \perp, \{q_2\})$, where δ is illustrated as follows



Exercises

What is the value of the transition function?

► $\delta(q_0, b, 1) = \emptyset$

► $\delta(q_0, a, \perp) = \{(q_0, 1\perp)\}$

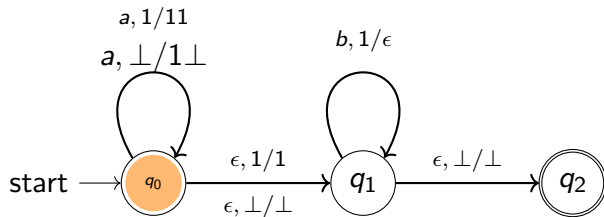
push

► $\delta(q_1, \epsilon, \perp) = \{(q_2, \perp)\}$

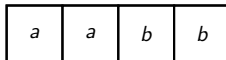
► $\delta(q_1, b, 1) = \{(q_1, \epsilon)\}$

pop

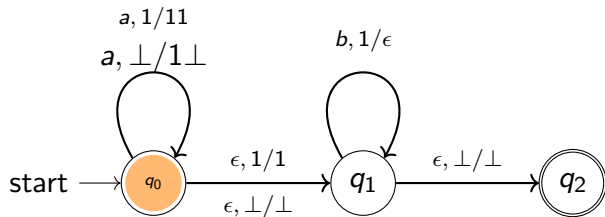
Running the PDA on $w = aabb$



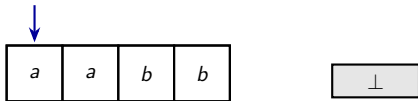
not started: state q_0



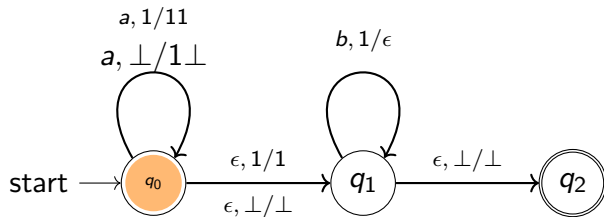
Running the PDA on $w = aabb$



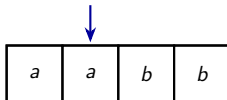
state q_0 **push 1**



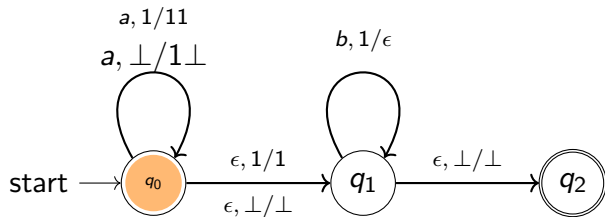
Running the PDA on $w = aabb$



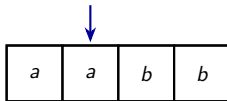
state q_0



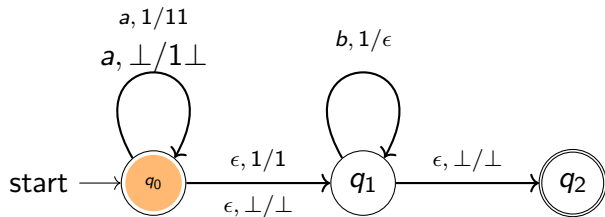
Running the PDA on $w = aabb$



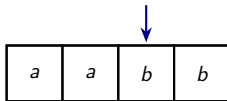
state q_0 push 1



Running the PDA on $w = aabb$

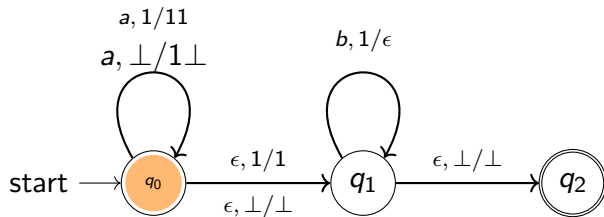


state q_0

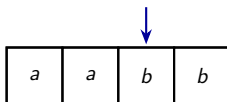


1
1
$\perp$

Running the PDA on $w = aabb$

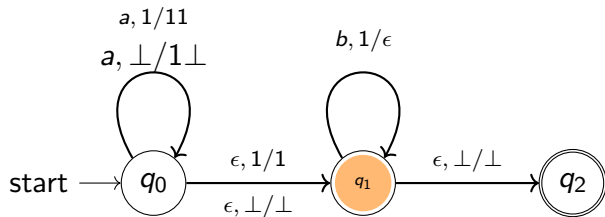


guess midpoint: $q_0 \rightarrow q_1, \epsilon, 1/1$

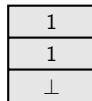
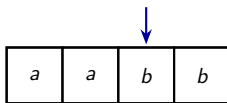


1
1
$\perp$

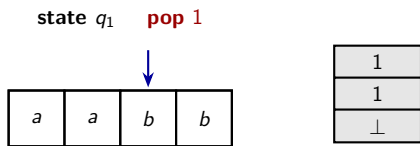
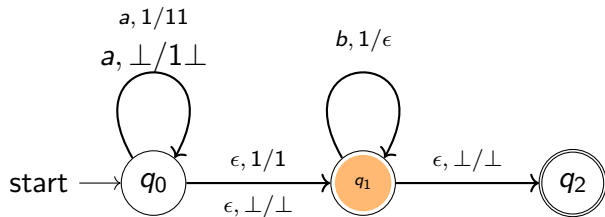
Running the PDA on $w = aabb$



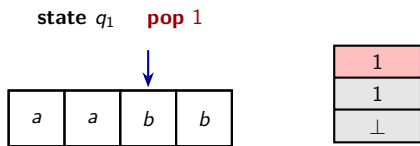
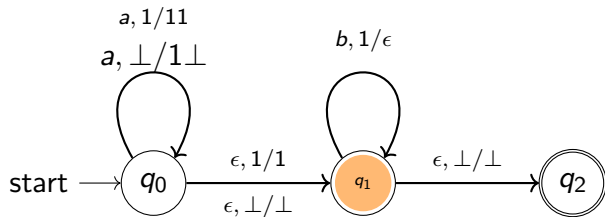
state q_1



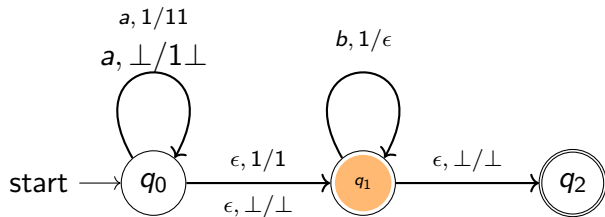
Running the PDA on $w = aabb$



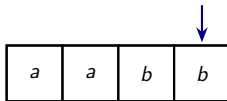
Running the PDA on $w = aabb$



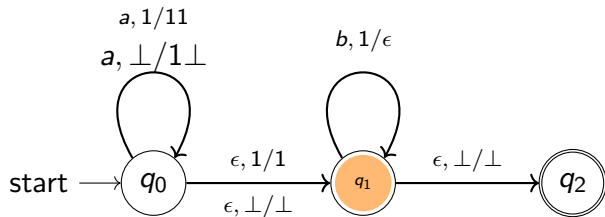
Running the PDA on $w = aabb$



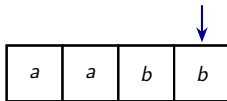
state q_1



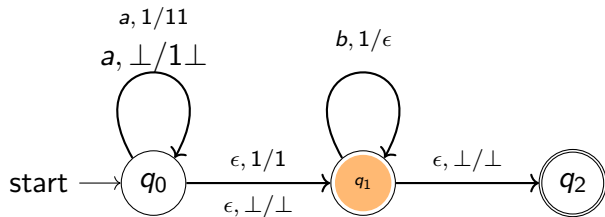
Running the PDA on $w = aabb$



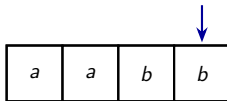
state q_1 **pop 1**



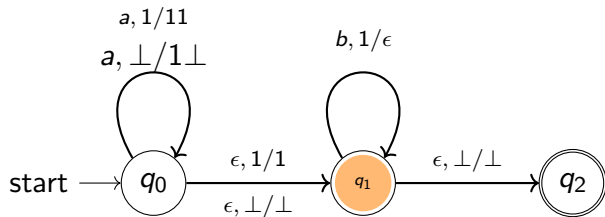
Running the PDA on $w = aabb$



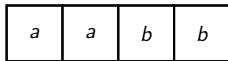
state q_1 **pop 1**



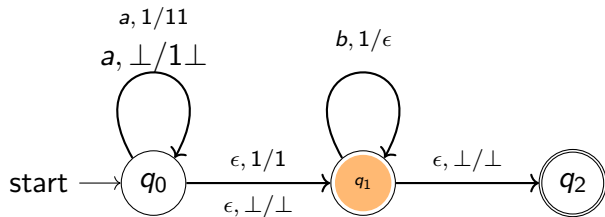
Running the PDA on $w = aabb$



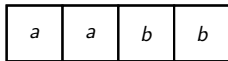
state q_1



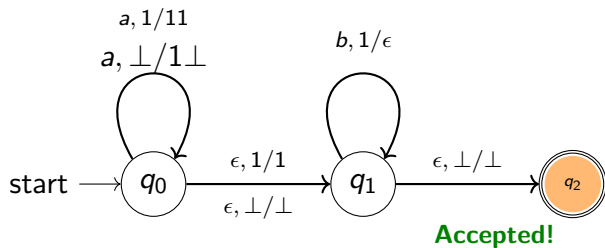
Running the PDA on $w = aabb$



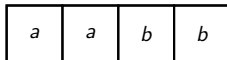
accept: $q_1 \rightarrow q_2, \epsilon, \perp/\perp$



Running the PDA on $w = aabb$

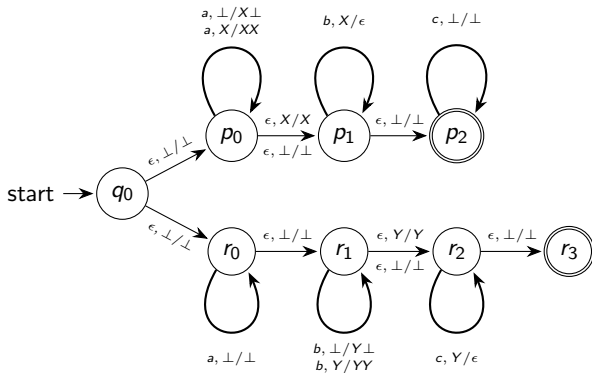


state q_2

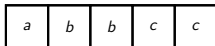
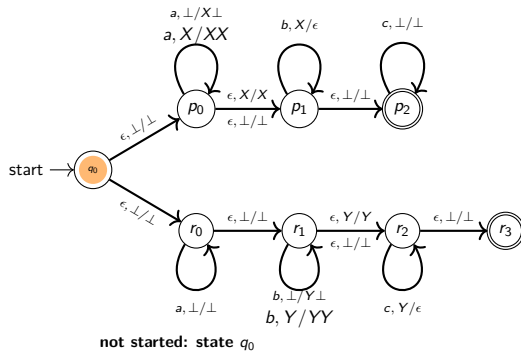


Another Example

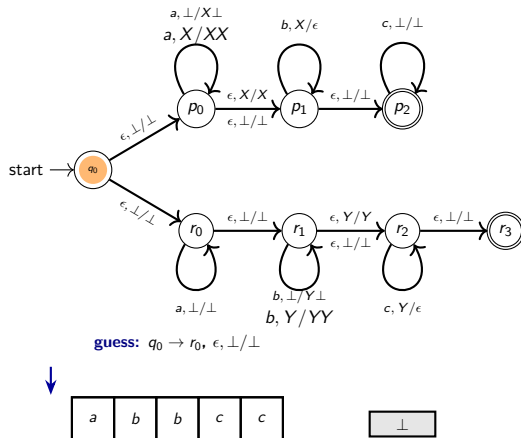
$$L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$$



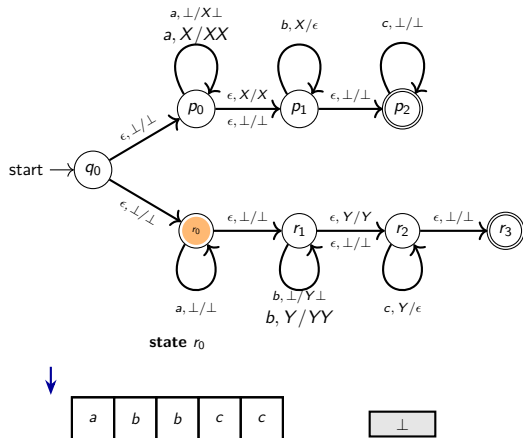
Running the PDA on $w = abbcc$



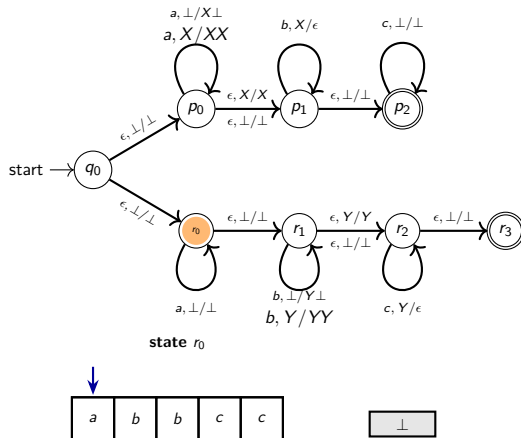
Running the PDA on $w = abbcc$



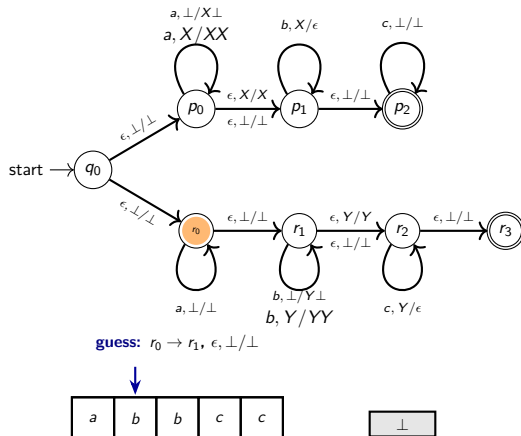
Running the PDA on $w = abbcc$



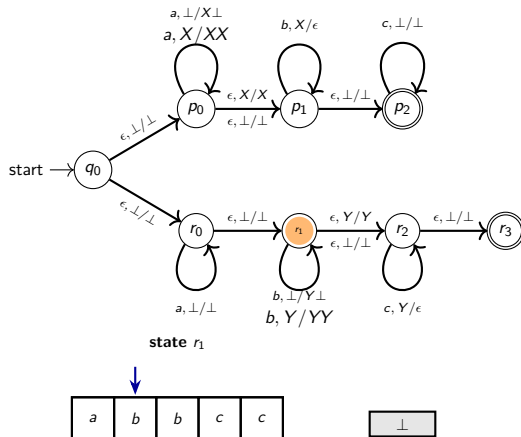
Running the PDA on $w = abbcc$



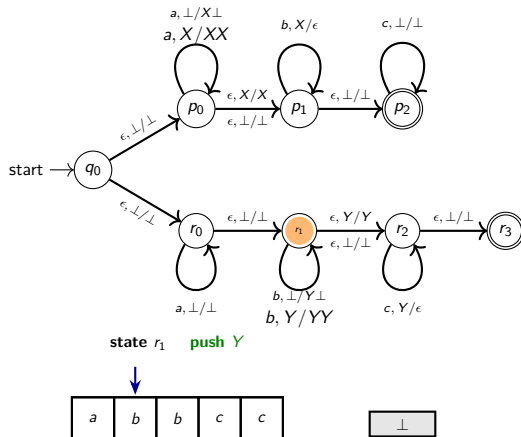
Running the PDA on $w = abbcc$



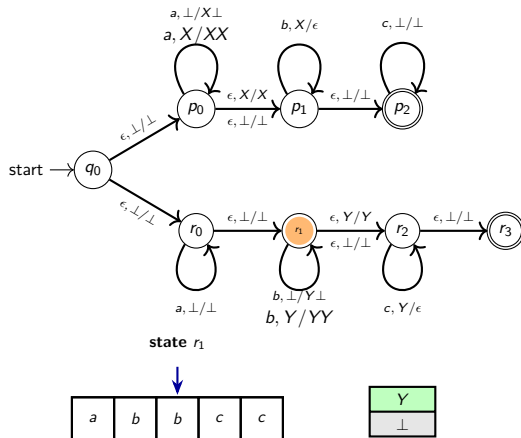
Running the PDA on $w = abbcc$



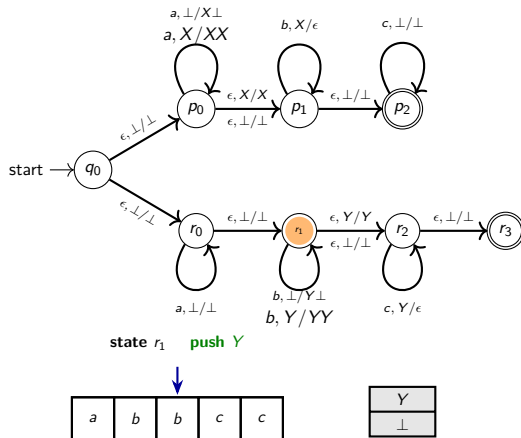
Running the PDA on $w = abbcc$



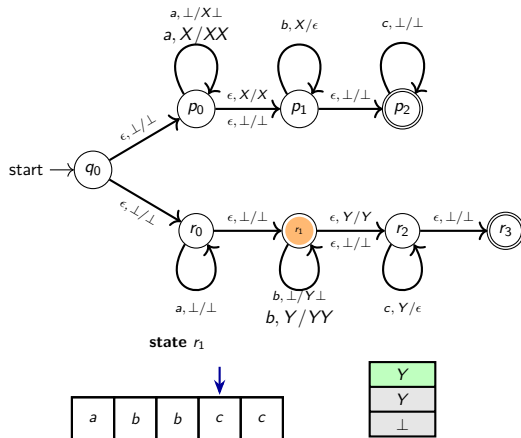
Running the PDA on $w = abbcc$



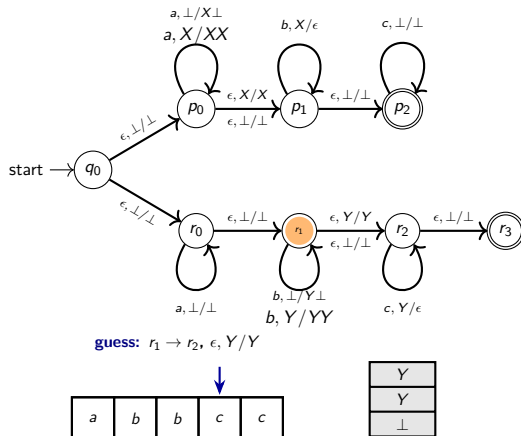
Running the PDA on $w = abbcc$



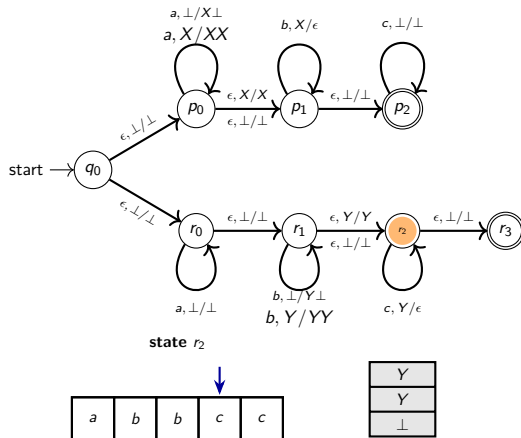
Running the PDA on $w = abbcc$



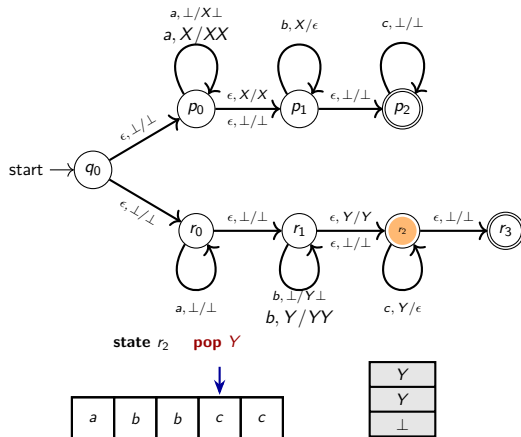
Running the PDA on $w = abbcc$



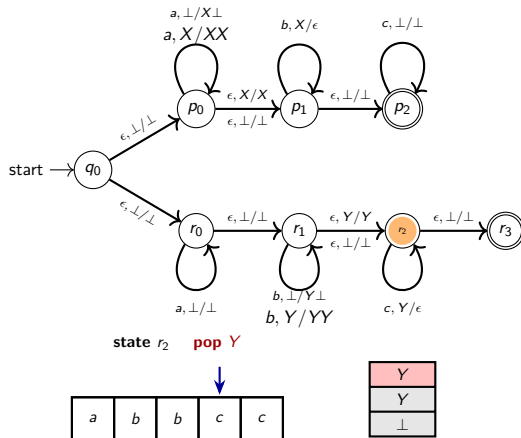
Running the PDA on $w = abbcc$



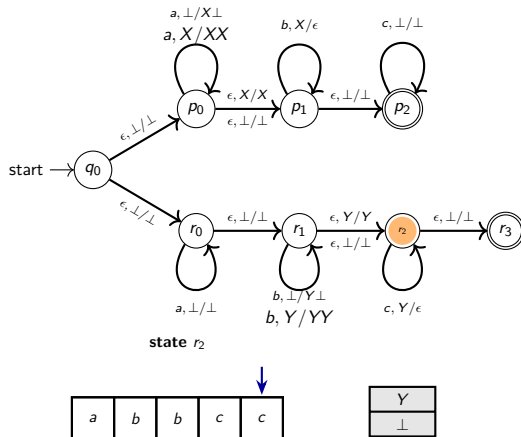
Running the PDA on $w = abbcc$



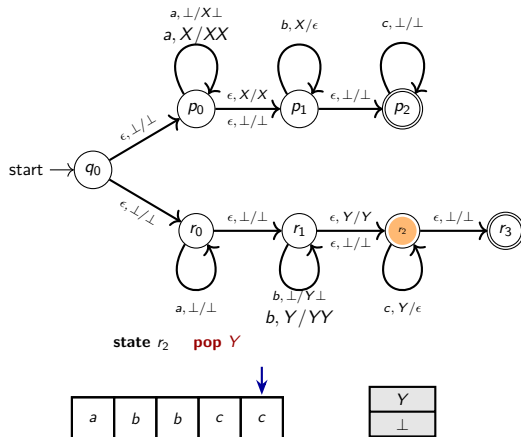
Running the PDA on $w = abbcc$



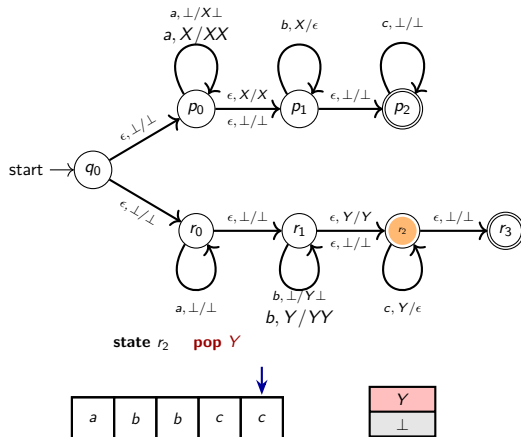
Running the PDA on $w = abbcc$



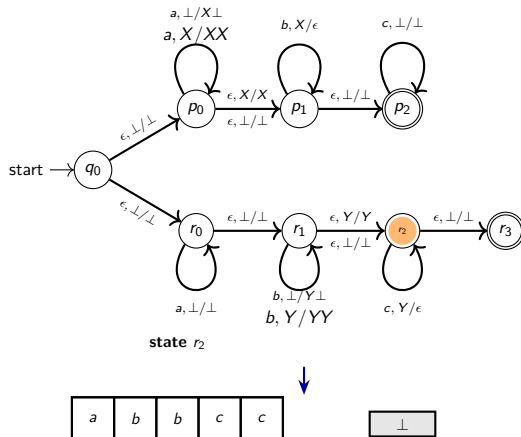
Running the PDA on $w = abbcc$



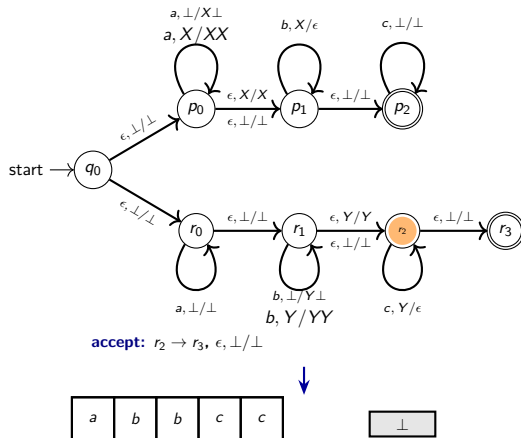
Running the PDA on $w = abbcc$



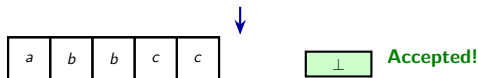
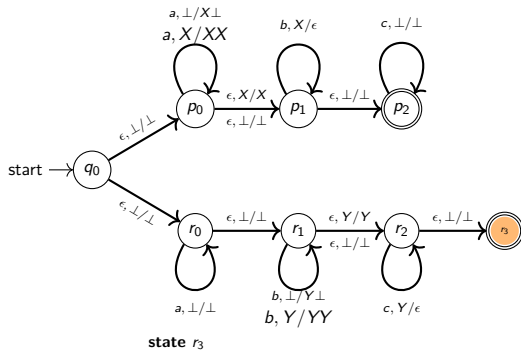
Running the PDA on $w = abbcc$



Running the PDA on $w = abbcc$



Running the PDA on $w = abbcc$



Configurations

- ▶ A **configuration** of A is an element of $Q \times \Sigma^* \times \Gamma^*$
- ▶ $(q, w, \gamma) :$ (current state, portion of input remaining to be read left to right, stack content written as a word with leftmost being top of stack)

Configurations

- ▶ A **configuration** of A is an element of $Q \times \Sigma^* \times \Gamma^*$
- ▶ $(q, w, \gamma) :$ (current state, portion of input remaining to be read left to right, stack content written as a word with leftmost being top of stack)
- ▶ On input w , initial configuration is $(q_0, w, \perp)$

Configurations

- ▶ A **configuration** of A is an element of $Q \times \Sigma^* \times \Gamma^*$
- ▶ (q, w, γ) : (current state, portion of input remaining to be read left to right, stack content written as a word with leftmost being top of stack)
- ▶ On input w , initial configuration is $(q_0, w, \perp)$
- ▶ The next configuration relation is denoted as $\vdash$
- ▶ If $(q, \gamma) \in \delta(p, a, Z)$ then $\forall y \in \Sigma^*, \beta \in \Gamma^*$,

$$(p, a \cdot y, Z\beta) \vdash (q, y, \gamma \cdot \beta)$$

Configurations

- ▶ A **configuration** of A is an element of $Q \times \Sigma^* \times \Gamma^*$
- ▶ $(q, w, \gamma) :$ (current state, portion of input remaining to be read left to right, stack content written as a word with leftmost being top of stack)
- ▶ On input w , initial configuration is $(q_0, w, \perp)$
- ▶ The next configuration relation is denoted as $\vdash$
- ▶ If $(q, \gamma) \in \delta(p, a, Z)$ then $\forall y \in \Sigma^*, \beta \in \Gamma^*$,

$$(p, a \cdot y, Z\beta) \vdash (q, y, \gamma \cdot \beta)$$

- ▶ If $(q, \gamma) \in \delta(p, \epsilon, Z)$ then $\forall y \in \Sigma^*, \beta \in \Gamma^*$,

$$(p, y, Z\beta) \vdash (q, y, \gamma \cdot \beta)$$

- ▶ This is called a **move**.

Configurations

- ▶ A **configuration** of A is an element of $Q \times \Sigma^* \times \Gamma^*$
- ▶ (q, w, γ) : (current state, portion of input remaining to be read left to right, stack content written as a word with leftmost being top of stack)
- ▶ On input w , initial configuration is $(q_0, w, \perp)$
- ▶ The next configuration relation is denoted as $\vdash$
- ▶ If $(q, \gamma) \in \delta(p, a, Z)$ then $\forall y \in \Sigma^*, \beta \in \Gamma^*$,

$$(p, a \cdot y, Z\beta) \vdash (q, y, \gamma \cdot \beta)$$

- ▶ If $(q, \gamma) \in \delta(p, \epsilon, Z)$ then $\forall y \in \Sigma^*, \beta \in \Gamma^*$,

$$(p, y, Z\beta) \vdash (q, y, \gamma \cdot \beta)$$

- ▶ This is called a **move**.

Acceptance

Acceptance by Final State

w is accepted by PDA A if $(q_0, w, \perp) \vdash^* (q, \epsilon, \gamma)$, where $q \in F$.

Acceptance

Acceptance by Final State

w is accepted by PDA A if $(q_0, w, \perp) \vdash^* (q, \epsilon, \gamma)$, where $q \in F$.

Acceptance by Empty Stack

w is accepted by PDA A if $(q_0, w, \perp) \vdash^* (q, \epsilon, \epsilon)$, where $q \in Q$.

Acceptance

Acceptance by Final State

w is accepted by PDA A if $(q_0, w, \perp) \vdash^* (q, \epsilon, \gamma)$, where $q \in F$.

Acceptance by Empty Stack

w is accepted by PDA A if $(q_0, w, \perp) \vdash^* (q, \epsilon, \epsilon)$, where $q \in Q$.

Language of a PDA

$L(A)$ = all words accepted by a PDA (how?)

Some notes

- ▶ Let NPDA denote non deterministic PDAs (what we have defined).

Some notes

- ▶ Let NPDA denote non deterministic PDAs (what we have defined).
- ▶ The initial stack symbol $\perp$ is needed just in the beginning. After that point, it can be popped out and pushed in like any other stack symbol. It need not stay at the bottom of the stack all the time, any symbol can be at the bottom of the stack after start.

Some notes

- ▶ Let NPDA denote non deterministic PDAs (what we have defined).
- ▶ The initial stack symbol $\perp$ is needed just in the beginning. After that point, it can be popped out and pushed in like any other stack symbol. It need not stay at the bottom of the stack all the time, any symbol can be at the bottom of the stack after start.
- ▶ If the stack is empty, **no transition** can be applied, and the machine is stuck.

Some notes

- ▶ Let NPDA denote non deterministic PDAs (what we have defined).
- ▶ The initial stack symbol $\perp$ is needed just in the beginning. After that point, it can be popped out and pushed in like any other stack symbol. It need not stay at the bottom of the stack all the time, any symbol can be at the bottom of the stack after start.
- ▶ If the stack is empty, **no transition** can be applied, and the machine is stuck.
- ▶ In acceptance by empty stack, we accept when the stack goes empty **after** applying a transition, assuming the input is completely read.

Acceptance conditions

Theorem

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ be a PDA.

1. If L is the language accepted of P by final state, there exists PDA P' such that its language accepted by empty stack is L

Acceptance conditions

Theorem

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ be a PDA.

1. If L is the language accepted of P by final state, there exists PDA P' such that its language accepted by empty stack is L
2. If L is the language accepted of P by empty stack, there exists PDA P' such that its language accepted by final state is L .